

8. Cost-based optimization

8.1. Overview

Lesson objectives

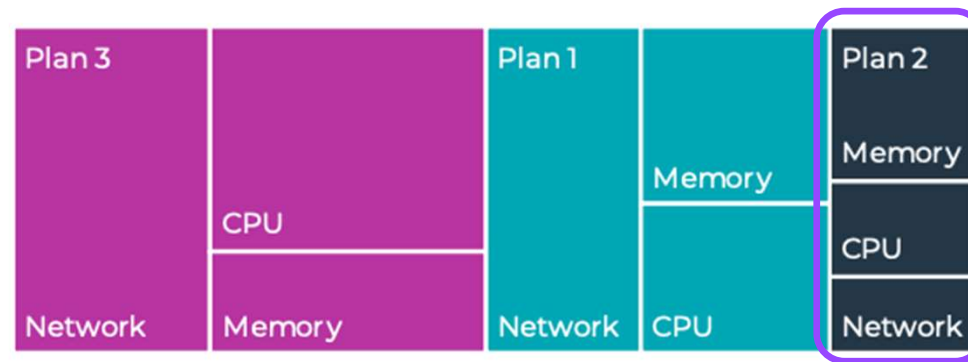
Cost-based optimization: Overview

1. Describe what cost-based optimization (CBO) is and why it matters.
2. Explain the role that table statistics play in CBO.
3. Use SQL to view and calculate tables statistics.

What is a cost-based optimizer?

- CBO evaluates multiple query execution plans and selects the most efficient one.
- It considers CPU time, network bandwidth, and memory requirements.
- Unlike a Rule-Based Optimizer (RBO), which follows static rules, CBO is adaptive and data-driven.

Plan with the lowest “cost” is chosen



Starburst CBO key components

- **Statistics Collection:** Gathers metadata like row count, low value, high value, and estimated distinct values.
- **Join Optimization:** Selects the best join strategy (broadcast vs. distributed).
- **Predicate Pushdown:** Filters data as early as possible to reduce scanned rows by pushing the predicate down to the underlying data source or to the lower layer in the data lake.
- **Partition Pruning:** Avoids scanning unnecessary partitions.
- **Reordering Joins:** Arranges tables in the most efficient join sequence.

Statistics collection: The foundation of CBO

- The CBO relies on accurate table statistics to estimate query costs.
- Table statistics:
 - Row count (# rows in table)
- Column statistics:
 - Data size (size of data that needs to be read)
 - Nulls fraction (fraction of null values)
 - Distinct value count (# distinct values)
 - Low value (smallest value in column)
 - High value (largest value in column)

```
trino> SHOW STATS FOR datalake.sales.lineitem;
```

column_name	data_size	distinct_values_count	nulls_fraction	row_count	low_value	high_value
orderkey	NULL	1547451.0	0.0	NULL	1	6000000
partkey	NULL	196274.0	0.0	NULL	1	200000
suppkey	NULL	10000.0	0.0	NULL	1	10000
linenumber	NULL	7.0	0.0	NULL	1	7
quantity	NULL	50.0	0.0	NULL	1.0	50.0
extendedprice	NULL	936107.0	0.0	NULL	901.0	104949.5
discount	NULL	11.0	0.0	NULL	0.0	0.1
tax	NULL	9.0	0.0	NULL	0.0	0.08
returnflag	6001215.0	3.0	0.0	NULL	NULL	NULL
linestatus	6001215.0	2.0	0.0	NULL	NULL	NULL
shipdate	NULL	2526.0	0.0	NULL	1992-01-02	1998-12-01
commitdate	NULL	2466.0	0.0	NULL	1992-01-31	1998-10-31
receiptdate	NULL	2554.0	0.0	NULL	1992-01-04	1998-12-31
shipinstruct	7.2006409E7	4.0	0.0	NULL	NULL	NULL
shipmode	2.5717034E7	7.0	0.0	NULL	NULL	NULL
comment	1.58997209E8	4668697.0	0.0	NULL	NULL	NULL
NULL	NULL	NULL	NULL	6001215.0	NULL	NULL

(17 rows)

View and calculate statistics

View statistics

```
SHOW STATS FOR table_name;
```

Calculate statistics

```
ANALYZE table_name WITH (  
    partitions = ARRAY[ARRAY['p2_value1', 'p2_value2']],  
    columns = ARRAY['col_1', 'col_2'])
```

Note: Identifying specific columns and/or partitions is optional, but recommended.

Query Optimization

Pushdown

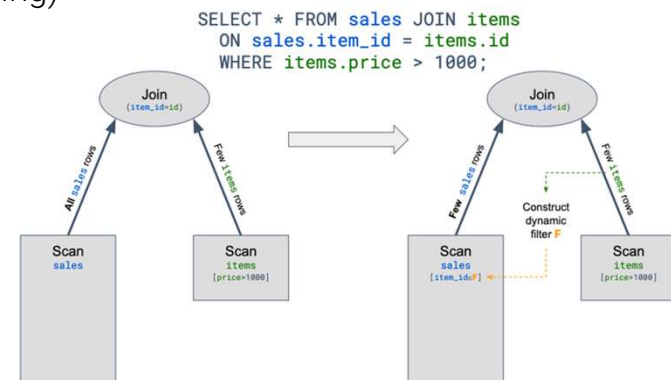
Move the filtering condition as close to the source of the data as possible.

- Some benefits:
 - improved overall query performance
 - reduced network traffic between Trino and the data source
 - reduced load on the remote data source
- Types:
 - Predicate
 - Aggregation
 - Join
 - Limit
 - Top-N

Dynamic Filtering

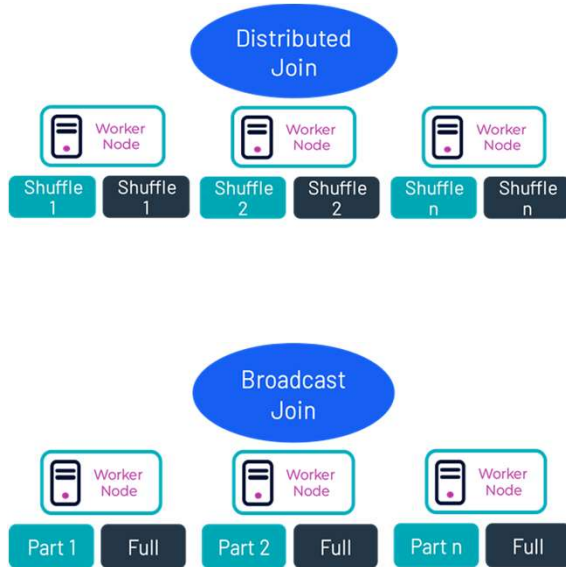
Significantly improve the performance of queries with selective joins by avoiding reading of data that would be filtered by join condition.

- Improve performance on federated queries
 - Better resource utilization for better data filtering
- Skip loading of partitions (dynamic partitioning pruning)

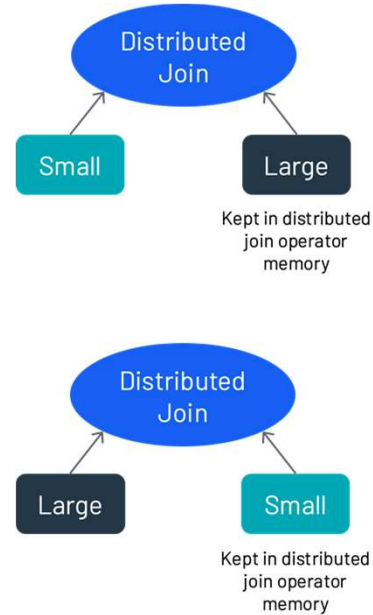


Query Improvements

Broadcast vs distributed



Join sides reordering



Join tree reordering

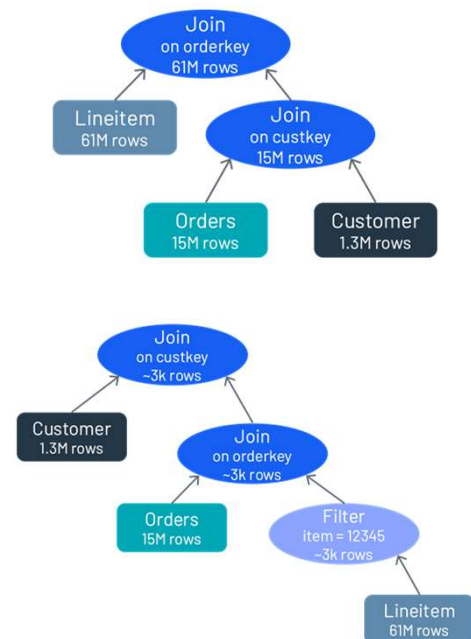


Table
A

Table
B

Lesson summary

Cost-based optimization: Overview

1. The cost-based optimizer (CBO) performs various transformations of a query plan to arrive at the most efficient variant.
2. CBO uses table statistics to identify the best plan.
3. SQL can be used to return the optimized plan with, or without runtime details, generated by the CBO.
4. Queries are typically optimized for pushdown, join optimizations, and dynamic filtering.

8. Cost-based optimization

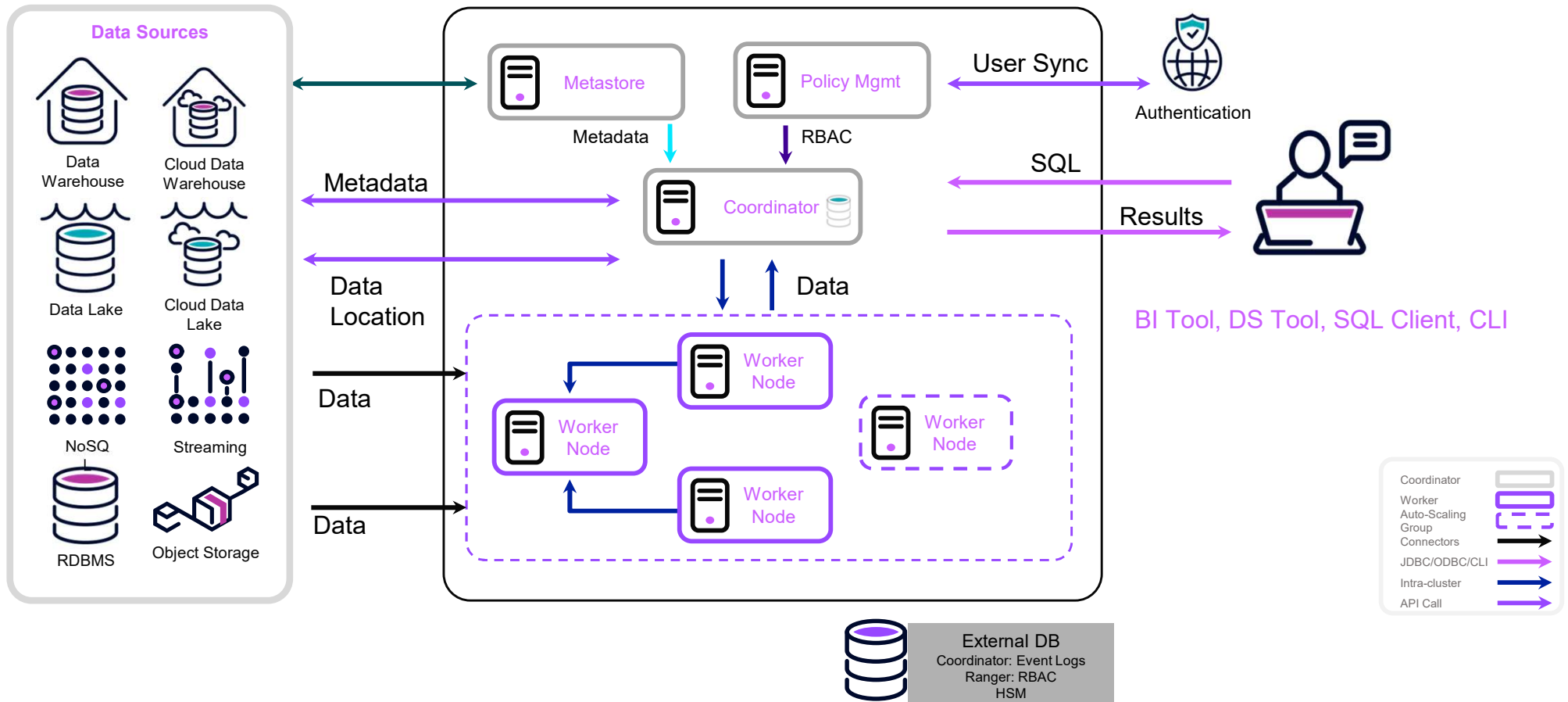
8.2. Query plan analysis

Lesson objectives

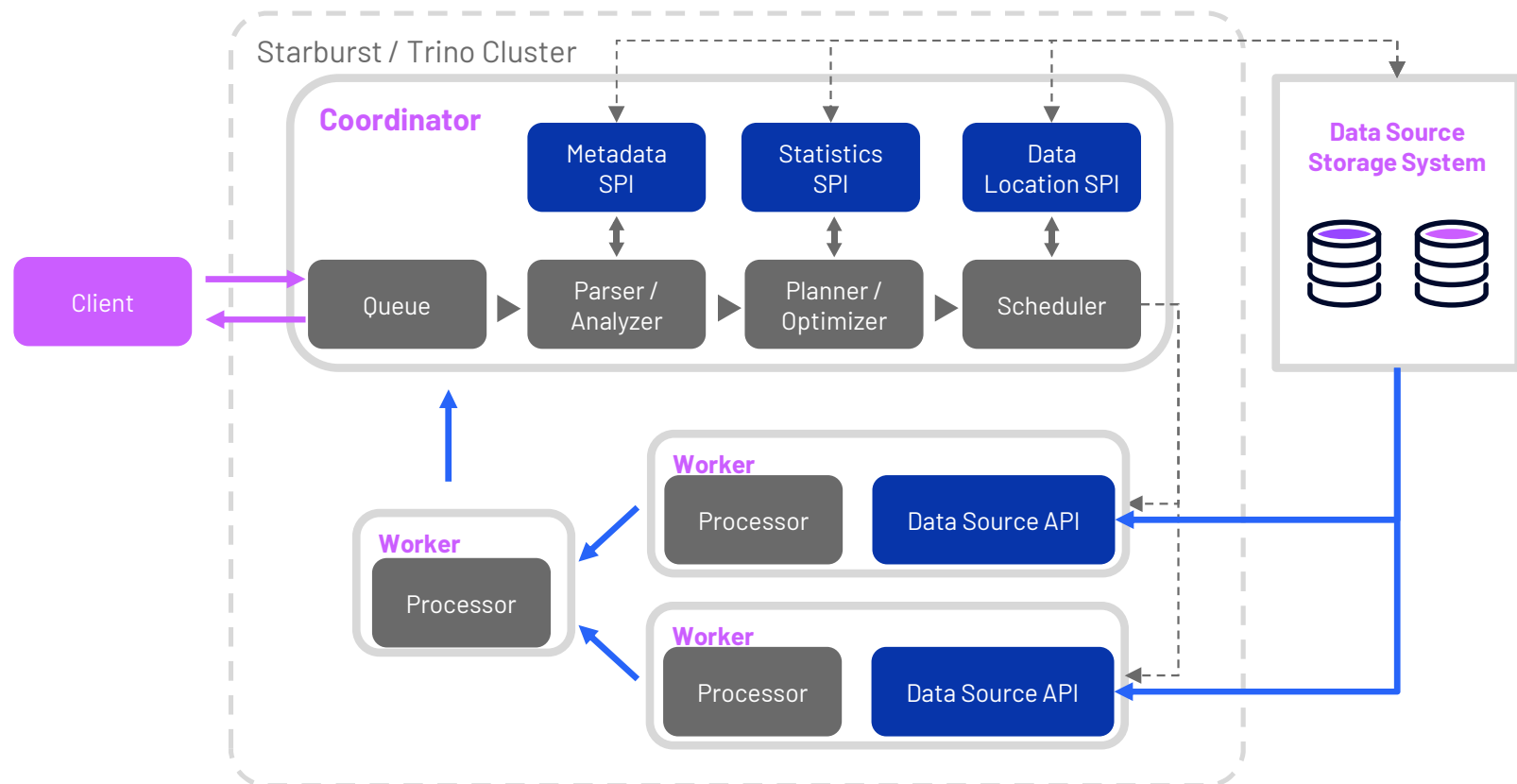
Cost-based optimization: Query plan analysis

1. Explain what query plans are and how they are achieved using Starburst architecture.
2. Describe the purpose of query plan analysis.
3. Analyze the typical output of an explain plan to determine value.
4. Explain how optimization is achieved and the mechanisms used to achieve it.

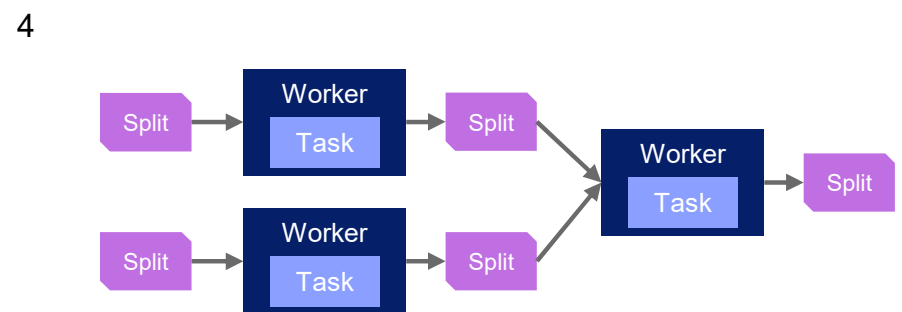
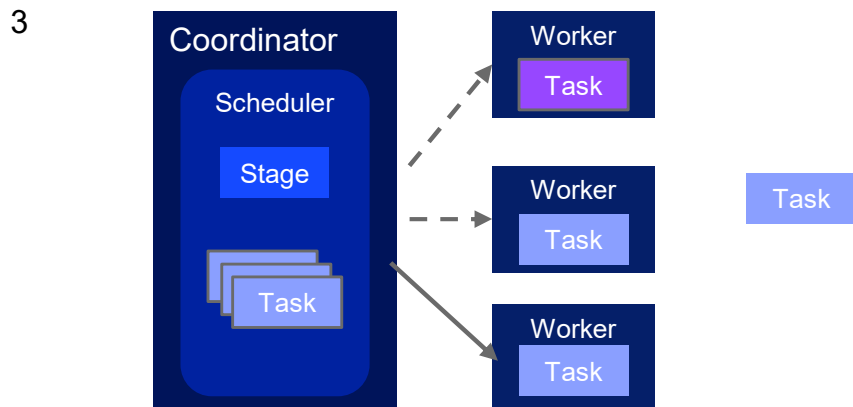
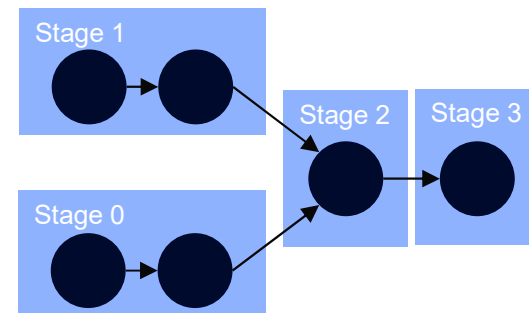
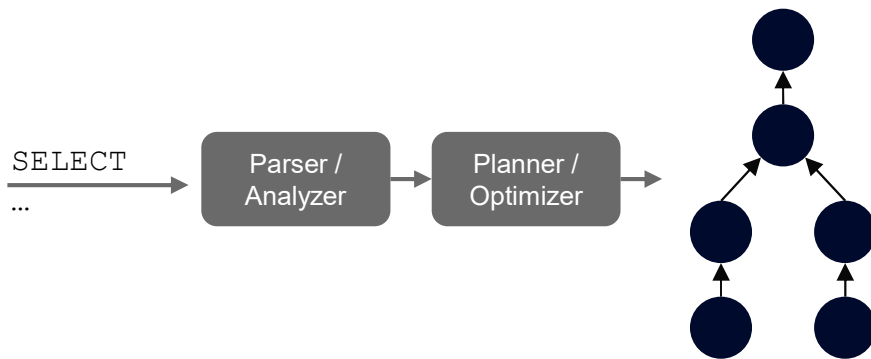
Starburst Logical Architecture



Starburst / Trino execution flow



Query Execution Model



Query Execution Model Nomenclature

- **Statement**: Trino executes ANSI-compatible SQL statements, which consist of clauses, expressions, and predicates.
- **Query**: a conversion of a parsed statement into a distributed query plan, which is then realized as a series of interconnected stages.
- **Stage**: When the query engine executes a query, it breaks up the execution into a hierarchy of stages.
- **Tasks**: a stage is implemented as a series of tasks distributed over a network of workers
- **Split**: sections of a larger data set. Stages retrieve data via splits from connectors as well as other stages higher up in a distributed query plan.
- **Driver**: Tasks contain one or more parallel drivers. Drivers act upon data and combine operators to produce output that is then aggregated by a task and then delivered to another task in another stage.
- **Operator**: An operator consumes, transforms, and produces data.
- **Exchange**: transfer data between Trino nodes for different stages of a query

EXPLAIN plan overview

EXPLAIN plans are key to identifying bottlenecks and optimizing joins, filters, and data access.

Let's take a look at the following:

- What tools are available for us to troubleshoot query issues?
- How can we use the tools to identify bottlenecks?
- How can we understand what is going on in the engine?
- What can we recommend to improve query performance?
- What information does Support need to troubleshoot issues?

What's an EXPLAIN Plan?

- Query Execution Plan
 - What logical tasks, in what order (table reads, joins, sorts, order by, etc)
 - What distribution type (how many workers to use per task)
- Uses Table Statistics
 - Estimates rows per task (influences join type & ordering)
- Output of CBO (cost-based optimizer)
 - Shows execution path / processing order for tasks
 - Shows optimizations (predicate push down, join reordering, etc)

```
EXPLAIN SELECT c.custkey, sum(l.extendedprice *  
l.discount) discount  
FROM customer c, orders o, lineitem l  
WHERE c.custkey = o.custkey  
      AND l.orderkey = o.orderkey  
GROUP BY c.custkey  
ORDER BY discount desc
```



```
Fragment 0 [SINGLE]  
  Output layout: [custkey_1, sum]  
  Output partitioning: SINGLE []  
  Output[columnNames = [custkey, discount]]  
  | Layout: [custkey_1:bigint, sum:double]  
  | Estimates: {rows: 99996 (1.72MB), cpu: 0, memory: 0B, network: 0B}  
  | custkey := custkey_1  
  | discount := sum  
  ↳ RemoteMerge[sourceFragmentIds = [1]]  
    Layout: [custkey_1:bigint, sum:double]  
  
Fragment 1 [ROUND_ROBIN]  
  Output layout: [custkey_1, sum]  
  Output partitioning: SINGLE []  
  LocalMerge[orderBy = [sum DESC NULLS LAST]]  
  | Layout: [custkey_1:bigint, sum:double]  
  | Estimates: {rows: 99996 (1.72MB), cpu: 0, memory: 0B, network: 0B}  
  ↳ PartialSort[orderBy = [sum DESC NULLS LAST]]  
    | Layout: [custkey_1:bigint, sum:double]  
    | Estimates: {rows: 99996 (1.72MB), cpu: 7, memory: 7, network: 7}  
    ↳ RemoteSource[sourceFragmentIds = [2]]  
      Layout: [custkey_1:bigint, sum:double]  
  
Fragment 2 [HASH]  
  Output layout: [custkey_1, sum]  
  Output partitioning: ROUND_ROBIN []  
  Aggregate[type = FINAL, keys = [custkey_1]]  
  | Layout: [custkey_1:bigint, sum:double]  
  | Estimates: {rows: 99996 (1.72MB), cpu: 103.02M, memory: 1.72MB, network: 0B}  
  | sum := sum(sum_6)  
  ↳ LocalExchange[partitioning = HASH, arguments = [custkey_1:bigint]]  
    | Layout: [custkey_1:bigint, sum_6:double]  
    | Estimates: {rows: 6001215 (103.02MB), cpu: 103.02M, memory: 0B, network: 0B}  
    ↳ RemoteSource[sourceFragmentIds = [3]]  
      Layout: [custkey_1:bigint, sum_6:double]
```

Generating an EXPLAIN Plan

- Prefix any query with the word 'EXPLAIN'
- Results in an execution plan
- Multiple variants:
 - EXPLAIN: Query plan before execution, plan structure + cost estimates
 - EXPLAIN ANALYZE: Plan structure + cost estimates + actual execution statistics

```
EXPLAIN ANALYZE
SELECT custkey, sum(totalprice) AS total
FROM orders
WHERE
    orderstatus = 'F' AND
    orderdate BETWEEN DATE '1995-01-01'
AND DATE '1995-12-31'
GROUP BY custkey
ORDER BY total DESC;
```

EXPLAIN Plan Output

Series of fragments:

- Representing logical sections of work to be performed in stages in cluster
- Work executed on single or multiple-nodes

```
Query: EXPLAIN SELECT custkey, sum(totalprice) AS total FROM orders WHERE orderstatus = 'F' AND orderdate between
DATE '1995-01-01' AND DATE '1995-12-31' GROUP BY custkey ORDER BY total DESC;
```

```
Fragment 0 [SINGLE]
  Output layout: [custkey, sum]
  Output partitioning: SINGLE []
  Output[columnNames = [custkey, total]]
  Layout: [custkey:bigint, sum:double]
  Estimates: {rows: 99595 (1.71MB), cpu: 0, memory: 0B, network: 0B}
  total := sum
  RemoteMerge[sourceFragmentIds = [1]]
    Layout: [custkey:bigint, sum:double]

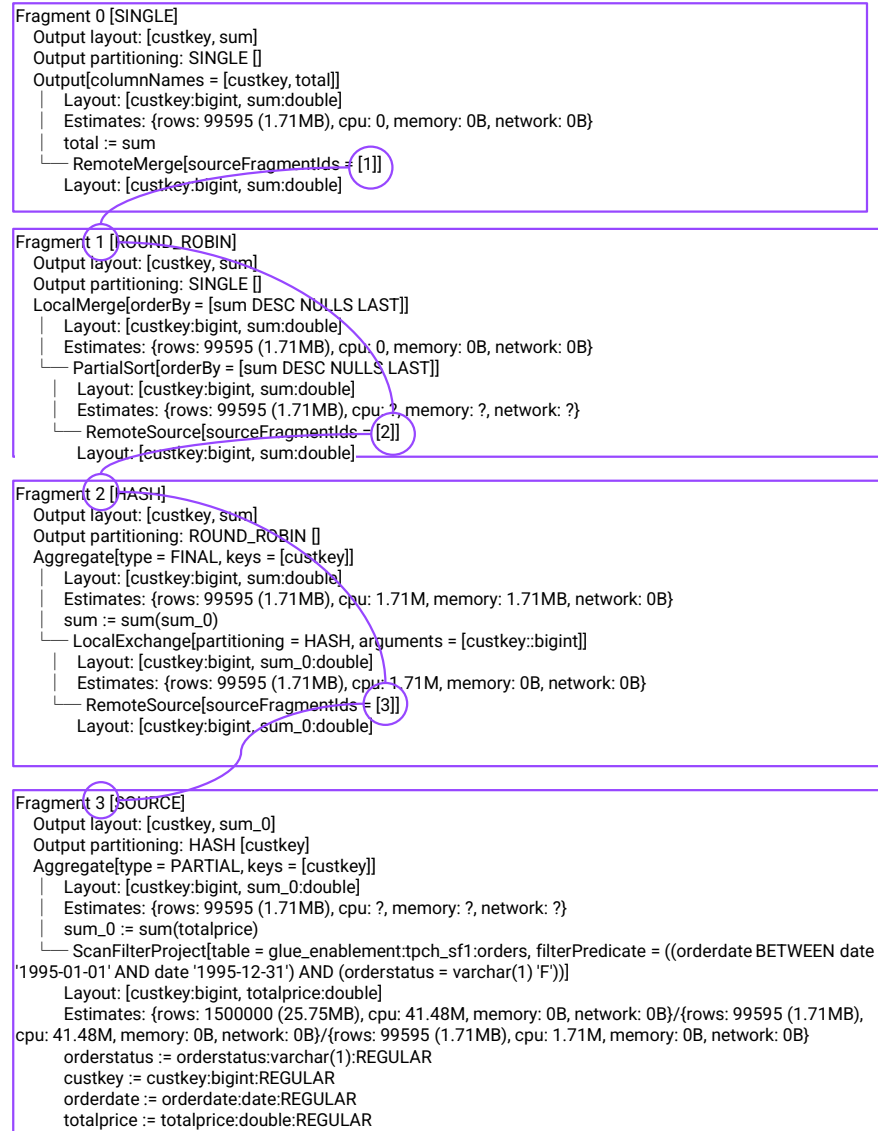
Fragment 1 [ROUND_ROBIN]
  Output layout: [custkey, sum]
  Output partitioning: SINGLE []
  LocalMerge[orderBy = [sum DESC NULLS LAST]]
    Layout: [custkey:bigint, sum:double]
    Estimates: {rows: 99595 (1.71MB), cpu: 0, memory: 0B, network: 0B}
  PartialSort[orderBy = [sum DESC NULLS LAST]]
    Layout: [custkey:bigint, sum:double]
    Estimates: {rows: 99595 (1.71MB), cpu: ?, memory: ?, network: ?}
  RemoteSource[sourceFragmentIds = [2]]
    Layout: [custkey:bigint, sum:double]

Fragment 2 [HASH]
  Output layout: [custkey, sum]
  Output partitioning: ROUND_ROBIN []
  Aggregate[type = FINAL, keys = [custkey]]
    Layout: [custkey:bigint, sum:double]
    Estimates: {rows: 99595 (1.71MB), cpu: 1.71M, memory: 1.71MB, network: 0B}
    sum := sum(sum_0)
  LocalExchange[partitioning = HASH, arguments = [custkey::bigint]]
    Layout: [custkey:bigint, sum_0:double]
    Estimates: {rows: 99595 (1.71MB), cpu: 1.71M, memory: 0B, network: 0B}
  RemoteSource[sourceFragmentIds = [3]]
    Layout: [custkey:bigint, sum_0:double]

Fragment 3 [SOURCE]
  Output layout: [custkey, sum_0]
  Output partitioning: HASH [custkey]
  Aggregate[type = PARTIAL, keys = [custkey]]
    Layout: [custkey:bigint, sum_0:double]
    Estimates: {rows: 99595 (1.71MB), cpu: ?, memory: ?, network: ?}
    sum_0 := sum(totalprice)
  ScanFilterProject[table = glue_enablement:tpch_sf1:orders, filterPredicate = ((orderdate BETWEEN date
'1995-01-01' AND date '1995-12-31') AND (orderstatus = varchar(1) 'F'))]
    Layout: [custkey:bigint, totalprice:double]
    Estimates: {rows: 1500000 (25.75MB), cpu: 41.48M, memory: 0B, network: 0B}/{rows: 99595 (1.71MB), cpu:
41.48M, memory: 0B, network: 0B}/{rows: 99595 (1.71MB), cpu: 1.71M, memory: 0B, network: 0B}
    orderstatus := orderstatus:varchar(1):REGULAR
    custkey := custkey:bigint:REGULAR
    orderdate := orderdate:date:REGULAR
    totalprice := totalprice:double:REGULAR
```

Exchanges

- Fragment ids indicate how data is transferred between fragments



EXPLAIN **VS** EXPLAIN ANALYZE output

```
Fragment 2 [HASH]
Output layout: [custkey, sum]
Output partitioning: ROUND_ROBIN []
Aggregate[type = FINAL, keys = [custkey]]
├── Layout: [custkey:bigint, sum:double]
│   ├── Estimates: {rows: 99595 (1.71MB), cpu: 1.71M, memory: 1.71MB, network: 0B}
│   └── sum := sum(sum_0)
└── LocalExchange[partitioning = HASH, arguments = [custkey::bigint]]
    ├── Layout: [custkey:bigint, sum_0:double]
    ├── Estimates: {rows: 99595 (1.71MB), cpu: 1.71M, memory: 0B, network: 0B}
    └── RemoteSource[sourceFragmentIds = [3]]
        └── Layout: [custkey:bigint, sum_0:double]

Fragment 3 [SOURCE]
Output layout: [custkey, sum_0]
Output partitioning: HASH [custkey]
Aggregate[type = PARTIAL, keys = [custkey]]
├── Layout: [custkey:bigint, sum_0:double]
│   ├── Estimates: {rows: 99595 (1.71MB), cpu: ?, memory: ?, network: ?}
│   └── sum_0 := sum(totalprice)
└── ScanFilterProject[table = glue_enablement:tpch_sf1:orders, filterPredicate = ((orderdate BETWEEN date '1995-01-01'
AND date '1995-12-31') AND (orderstatus = varchar(1) 'F'))]
    ├── Layout: [custkey:bigint, totalprice:double]
    ├── Estimates: {rows: 1500000 (25.75MB), cpu: 41.48M, memory: 0B, network: 0B}/{rows: 99595 (1.71MB), cpu: 41.48M,
memory: 0B, network: 0B}/{rows: 99595 (1.71MB), cpu: 1.71M, memory: 0B, network: 0B}
    ├── orderstatus := orderstatus:varchar(1):REGULAR
    ├── custkey := custkey:bigint:REGULAR
    ├── orderdate := orderdate:date:REGULAR
    └── totalprice := totalprice:double:REGULAR
```

EXPLAIN Plan

- EXPLAIN is useful for:
 - A long running query when you just want the Query Plan
 - Out Of Memory issues for queries

```
Fragment 2 [HASH]
CPU: 21.52ms, Scheduled: 27.91ms, Blocked 2.36m (Input: 1.16m, Output: 0.00ns), Input: 40371 rows (709.65kB); per task: avg.: 20185.50
std.dev.: 37.50, Output: 37377 rows (657.02kB)
Peak Memory: 10.17MB, Tasks count: 2; per task: max: 9.80MB
Output layout: [custkey, sum]
Output partitioning: ROUND_ROBIN []
Aggregate[type = FINAL, keys = [custkey]]
├── Layout: [custkey:bigint, sum:double]
│   ├── Estimates: {rows: 99595 (1.71MB), cpu: 1.71M, memory: 1.71MB, network: 0B}
│   ├── CPU: 13.00ms (2.80%), Scheduled: 18.00ms (1.19%), Blocked: 0.00ns (0.00%), Output: 37377 rows (657.02kB)
│   ├── Input avg.: 630.80 rows, Input std.dev.: 3.94%
│   └── sum := sum(sum_0)
└── LocalExchange[partitioning = HASH, arguments = [custkey::bigint]]
    ├── Layout: [custkey:bigint, sum_0:double]
    ├── Estimates: {rows: 99595 (1.71MB), cpu: 1.71M, memory: 0B, network: 0B}
    ├── CPU: 2.00ms (0.43%), Scheduled: 2.00ms (0.13%), Blocked: 1.18m (33.21%), Output: 40371 rows (709.65kB)
    ├── Input avg.: 630.80 rows, Input std.dev.: 453.30%
    └── RemoteSource[sourceFragmentIds = [3]]
        ├── Layout: [custkey:bigint, sum_0:double]
        ├── CPU: 0.00ns (0.00%), Scheduled: 0.00ns (0.00%), Blocked: 1.16m (32.44%), Output: 40371 rows (709.65kB)
        └── Input avg.: 630.80 rows, Input std.dev.: 453.30%

Fragment 3 [SOURCE]
CPU: 421.95ms, Scheduled: 1.46s, Blocked 0.00ns (Input: 0.00ns, Output: 0.00ns), Input: 1500000 rows (41.48MB); per task: avg.: 750000.00
std.dev.: 480379.00, Output: 40371 rows (709.65kB)
Peak Memory: 676.63kB, Tasks count: 2; per task: max: 13.03MB
Amount of input data processed by the workers for this stage might be skewed
Output layout: [custkey, sum_0]
Output partitioning: HASH [custkey]
Aggregate[type = PARTIAL, keys = [custkey]]
├── Layout: [custkey:bigint, sum_0:double]
│   ├── Estimates: {rows: 99595 (1.71MB), cpu: ?, memory: ?, network: ?}
│   ├── CPU: 14.00ms (3.01%), Scheduled: 14.00ms (0.93%), Blocked: 0.00ns (0.00%), Output: 40371 rows (709.65kB)
│   ├── Input avg.: 24041.00 rows, Input std.dev.: 63.75%
│   └── sum_0 := sum(totalprice)
└── ScanFilterProject[table = glue_enablement:tpch_sf1:orders, filterPredicate = ((orderdate BETWEEN date '1995-01-01' AND date '1995-12-31')
AND (orderstatus = varchar(1) 'F'))]
    ├── Layout: [custkey:bigint, totalprice:double]
    ├── Estimates: {rows: 1500000 (25.75MB), cpu: 41.48M, memory: 0B, network: 0B}/{rows: 99595 (1.71MB), cpu: 41.48M, memory: 0B, network:
0B}/{rows: 99595 (1.71MB), cpu: 1.71M, memory: 0B, network: 0B}
    ├── CPU: 408.00ms (87.74%), Scheduled: 1.45s (95.83%), Blocked: 0.00ns (0.00%), Output: 48082 rows (845.19kB)
    ├── Input avg.: 750000.00 rows, Input std.dev.: 64.05%
    ├── orderstatus := orderstatus:varchar(1):REGULAR
    ├── custkey := custkey:bigint:REGULAR
    ├── orderdate := orderdate:date:REGULAR
    ├── totalprice := totalprice:double:REGULAR
    └── Input: 1500000 rows (41.48MB), Filtered: 96.79%, Physical input: 16.21MB, Physical input time: 1.26s
```

EXPLAIN ANALYZE Plan

Distribution

- Work in fragments is executed in single or fixed number of nodes
- Fragment type indicates distribution of work
- Source fragments access data from source
- Hash and Round Robin are distributed across nodes

SINGLE
SOURCE
HASH
ROUND ROBIN

Fragment 3: **SOURCE**

CPU: 421.95ms, Scheduled: 1.46s, Blocked 0.00ns (Input: 0.00ns, Output: 0.00ns), Input: 1500000 rows (41.48MB); per task: avg.: 750000.00
std.dev.: 480379.00, Output: 40371 rows (709.65kB)
Peak Memory: 676.63kB, Tasks count: 2; per task: max: 13.03MB
Amount of input data processed by the workers for this stage might be skewed
Output layout: [custkey, sum_0]
Output partitioning: HASH [custkey]
Aggregate[type = PARTIAL, keys = [custkey]]
| Layout: [custkey:bigint, sum_0:double]
| Estimates: {rows: 99595 (1.71MB), cpu: ?, memory: ?, network: ?}
| CPU: 14.00ms (3.01%), Scheduled: 14.00ms (0.93%), Blocked: 0.00ns (0.00%), Output: 40371 rows (709.65kB)
| Input avg.: 24041.00 rows, Input std.dev.: 63.75%
| sum_0 := sum(totalprice)
| **ScanFilterProject[table = glue_enablement:tpch_sf1:orders, filterPredicate = ((orderdate BETWEEN date '1995-01-01' AND date '1995-12-31')**
AND (orderstatus = varchar(1) 'F'))]
| Layout: [custkey:bigint, totalprice:double]
| Estimates: {rows: 1500000 (25.75MB), cpu: 41.48M, memory: 0B, network: 0B}/{rows: 99595 (1.71MB), cpu: 41.48M, memory: 0B, network: 0B}/{rows: 99595 (1.71MB), cpu: 1.71M, memory: 0B, network: 0B}
| CPU: 408.00ms (87.74%), Scheduled: 1.45s (95.83%), Blocked: 0.00ms (0.00%), Output: 48082 rows (845.19kB)
| Input avg.: 750000.00 rows, Input std.dev.: 64.05%
| orderstatus := orderstatus:varchar(1):REGULAR
| custkey := custkey:bigint:REGULAR
| orderdate := orderdate:date:REGULAR
| totalprice := totalprice:double:REGULAR
| Input: 1500000 rows (41.48MB), Filtered: 96.79%, Physical input: 16.21MB, Physical input time: 1.26s

Fragment structure

Global
fragment
details

Fragment 3 [SOURCE]

CPU: 421.95ms, Scheduled: 1.46s, Blocked 0.00ns (Input: 0.00ns, Output: 0.00ns), Input: 1500000 rows (41.48MB); per task: avg.: 750000.00 std.dev.: 480379.00, Output: 40371 rows (709.65kB)
Peak Memory: 676.63kB, Tasks count: 2; per task: max: 13.03MB
Amount of input data processed by the workers for this stage might be skewed
Output layout: [custkey, sum_0]
Output partitioning: HASH [custkey]

Aggregate[type = PARTIAL, keys = [custkey]]

Layout: [custkey:bigint, sum_0:double]
Estimates: {rows: 99595 (1.71MB), cpu: ?, memory: ?, network: ?}
CPU: 14.00ms (3.01%), Scheduled: 14.00ms (0.93%), Blocked: 0.00ns (0.00%), Output: 40371 rows (709.65kB)
Input avg.: 24041.00 rows, Input std.dev.: 63.75%
sum_0 := sum(totalprice)

Plan node
details

ScanFilterProject[table = glue_enablement:tpch_sf1:orders, filterPredicate = ((orderdate BETWEEN date '1995-01-01' AND date '1995-12-31') AND (orderstatus = varchar(1) 'F'))]
Layout: [custkey:bigint, totalprice:double]
Estimates: {rows: 1500000 (25.75MB), cpu: 41.48M, memory: 0B, network: 0B}/{rows: 99595 (1.71MB), cpu: 41.48M, memory: 0B, network: 0B}/{rows: 99595 (1.71MB), cpu: 1.71M, memory: 0B, network: 0B}
CPU: 408.00ms (87.74%), Scheduled: 1.45s (95.83%), Blocked: 0.00ns (0.00%), Output: 48082 rows (845.19kB)
Input avg.: 750000.00 rows, Input std.dev.: 64.05%
orderstatus := orderstatus:varchar(1):REGULAR
custkey := custkey:bigint:REGULAR
orderdate := orderdate:date:REGULAR
totalprice := totalprice:double:REGULAR
Input: 1500000 rows (41.48MB), Filtered: 96.79%, Physical input: 16.21MB, Physical input time: 1.26s

Plan node
details

Source Fragment: Column pruning

EXPLAIN ANALYZE Query:

```
EXPLAIN ANALYZE
SELECT orderstatus, sum(totalprice)
FROM orders
GROUP BY orderstatus
```

Table DDL:

```
TABLE orders (
  orderkey bigint,
  custkey bigint,
  orderstatus varchar(1),
  totalprice double,
  orderdate date,
  orderpriority varchar(15),
  clerk varchar(15),
  shippriority integer,
  comment varchar(79))
```

Fragment 2 [SOURCE]

CPU: 195.48ms, Scheduled: 752.63ms, Blocked 0.00ns (Input: 0.00ns, Output: 0.00ns), Input: 1500000 rows (21.46MB); per task: avg.: 750000.00 std.dev.: 480379.00, Output: 6 rows (90B)

Peak Memory: 6.80MB, Tasks count: 2; per task: max: 6.99MB

Amount of input data processed by the workers for this stage might be skewed

Output layout: [orderstatus, sum_0]

Output partitioning: HASH [orderstatus]

Aggregate[type = PARTIAL, keys = [orderstatus]]

| Layout: [orderstatus:varchar(1), sum_0:double]

| Estimates: {rows: 1500000 (21.46MB), cpu: ?, memory: ?, network: ?}

| CPU: 87.00ms (44.62%), Scheduled: 87.00ms (11.57%), Blocked: 0.00ns (0.00%), Output: 6 rows (90B)

| Input avg.: 750000.00 rows, Input std.dev.: 64.05%

| sum_0 := sum(totalprice)

└─ TableScan[table = glue_enablement:tpch_sf1:orders]

Layout: [orderstatus:varchar(1), totalprice:double]

Estimates: {rows: 1500000 (21.46MB), cpu: 21.46M, memory: 0B, network: 0B}

CPU: 108.00ms (55.38%), Scheduled: 665.00ms (88.43%), Blocked: 0.00ns (0.00%), Output: 1500000 rows (21.46MB)

Input avg.: 750000.00 rows, Input std.dev.: 64.05%

orderstatus:= orderstatus:varchar(1):REGULAR

totalprice := totalprice:double:REGULAR

Input: 1500000 rows (21.46MB), Physical input: 11.28MB, Physical input time: 606.90ms

Estimates

- Cost-based optimizer relies on statistics being available from columns in queries.
- Only need stats for:
 - join keys
 - Columns filtered on
- Estimates sections show stats that are available
 - If no stats are available, numbers will be question marks
- Collected stats include:
 - Total size in bytes of columns
 - Number of distinct values
 - Fraction of nulls
 - Number of rows
 - Min/Max value
- Stats collected with Hive connector by running `ANALYZE` on tables
- Stats for databases collected by database

```
Fragment 3 [SOURCE]
  CPU: 421.95ms, Scheduled: 1.46s, Blocked 0.00ns (Input: 0.00ns, Output: 0.00ns), Input: 1500000
rows (41.48MB); per task: avg.: 750000.00 std.dev.: 480379.00, Output: 40371 rows (709.65kB)
  Peak Memory: 676.63kB, Tasks count: 2; per task: max: 13.03MB
  Amount of input data processed by the workers for this stage might be skewed
  Output layout: [custkey, sum_0]
  Output partitioning: HASH [custkey]
  Aggregate[type = PARTIAL, keys = [custkey]]
  | Layout: [custkey:bigint, sum_0:double]
  | Estimates: {rows: 99595 (1.71MB), cpu: ?, memory: ?, network: ?}
  | CPU: 14.00ms (3.01%), Scheduled: 14.00ms (0.93%), Blocked: 0.00ns (0.00%), Output: 40371
rows (709.65kB)
  | Input avg.: 24041.00 rows, Input std.dev.: 63.75%
  | sum_0 := sum(totalprice)
  | └─ ScanFilterProject[table = glue_enablement:tpch_sf1:orders, filterPredicate = ((orderdate
BETWEEN date '1995-01-01' AND date '1995-12-31') AND (orderstatus = varchar(1) 'F'))]
  | Layout: [custkey:bigint, totalprice:double]
  | Estimates: {rows: 1500000 (25.75MB), cpu: 41.48M, memory: 0B, network: 0B}/{rows: 99595
(1.71MB), cpu: 41.48M, memory: 0B, network: 0B}/{rows: 99595 (1.71MB), cpu: 1.71M, memory: 0B,
network: 0B}
  CPU: 408.00ms (87.74%), Scheduled: 1.45s (95.83%), Blocked: 0.00ns (0.00%), Output: 48082
rows (845.19kB)
  Input avg.: 750000.00 rows, Input std.dev.: 64.05%
  orderstatus := orderstatus:varchar(1):REGULAR
  custkey := custkey:bigint:REGULAR
  orderdate := orderdate:date:REGULAR
  totalprice := totalprice:double:REGULAR
  Input: 1500000 rows (41.48MB), Filtered: 96.79%, Physical input: 16.21MB, Physical input
time: 1.26s
```

'Layout' rows

- Columns passed as output from one section to the next
- Provides tracking of columns from beginning to final stages of query execution
- Internal names can be assigned to columns during execution

```
Fragment 3 [SOURCE]
  CPU: 421.95ms, Scheduled: 1.46s, Blocked: 0.00ns (Input: 0.00ns, Output: 0.00ns), Input: 1500000 rows
  (41.48MB); per task: avg.: 750000.00 std.dev.: 480379.00, Output: 40371 rows (709.65kB)
  Peak Memory: 676.63kB, Tasks count: 2; per task: max: 13.03MB
  Amount of input data processed by the workers for this stage might be skewed
  Output layout: [custkey, sum_0]
  Output partitioning: HASH [custkey]
  Aggregate[type = PARTIAL, keys = [custkey]]
    | Layout: [custkey:bigint, sum_0:double]
    | Estimates: {rows: 99595 (1.71MB), cpu: ?, memory: ?, network: ?}
    | CPU: 14.00ms (3.01%), Scheduled: 14.00ms (0.93%), Blocked: 0.00ns (0.00%), Output: 40371 rows
  (709.65kB)
    | Input avg.: 24041.00 rows, Input std.dev.: 63.75%
    | sum_0 := sum(totalprice)
    | ScanFilterProject[table = glue_enablement:tpch_sf1:orders, filterPredicate = ((orderdate BETWEEN date
  '1995-01-01' AND date '1995-12-31') AND (orderstatus = varchar(1) 'F'))]
    | Layout: [custkey:bigint, totalprice:double]
    | Estimates: {rows: 1500000 (25.75MB), cpu: 41.48M, memory: 0B, network: 0B}/{rows: 99595 (1.71MB), cpu:
  41.48M, memory: 0B, network: 0B}/{rows: 99595 (1.71MB), cpu: 1.71M, memory: 0B, network: 0B}
    | CPU: 408.00ms (87.74%), Scheduled: 1.45s (95.83%), Blocked: 0.00ns (0.00%), Output: 48082 rows
  (845.19kB)
    | Input avg.: 750000.00 rows, Input std.dev.: 64.05%
    | orderstatus := orderstatus:varchar(1):REGULAR
    | custkey := custkey:bigint:REGULAR
    | orderdate := orderdate:date:REGULAR
    | totalprice := totalprice:double:REGULAR
    | Input: 1500000 rows (41.48MB), Filtered: 96.79%, Physical input: 16.21MB, Physical input time: 1.26s
```

Performance stats

- **Scheduled** time = total amount of time work was scheduled. Accounts for:
 - CPU time
 - Reading data from source (ex: tables, files)
 - Transferring data across network
- **CPU** time= portion of Scheduled time when CPU was active
- Difference in metrics shows time waiting for data to come into cluster
- (%) = percentage of total query time

Fragment 3 [SOURCE]

CPU: 421.95ms, Scheduled: 1.46s, Blocked: 0.00ns (Input: 0.00ns, Output: 0.00ns), Input: 1500000 rows (41.48MB); per task: avg.: 750000.00 std.dev.: 480379.00, Output: 40371 rows (709.65kB)

Peak Memory: 676.63kB, Tasks count: 2; per task: max: 13.03MB

Amount of input data processed by the workers for this stage might be skewed

Output layout: [custkey, sum_0]

Output partitioning: HASH [custkey]

Aggregate[type = PARTIAL, keys = [custkey]]

Layout: [custkey:bigint, sum_0:double]

Estimates: {rows: 99595 (1.71MB), cpu: ?, memory: ?, network: ?}

CPU: 14.00ms (3.01%), Scheduled: 14.00ms (0.93%), Blocked: 0.00ns (0.00%), Output: 40371 rows (709.65kB)

Input avg.: 24041.00 rows, Input std.dev.: 63.75%

sum_0 := sum(totalprice)

ScanFilterProject[table = glue_enablement:tpch_sf1:orders, filterPredicate = ((orderdate BETWEEN date '1995-01-01' AND date '1995-12-31') AND (orderstatus = varchar(1) 'F'))]

Layout: [custkey:bigint, totalprice:double]

Estimates: {rows: 1500000 (25.75MB), cpu: 41.48M, memory: 0B, network: 0B}/{rows: 99595 (1.71MB), cpu: 41.48M, memory: 0B, network: 0B}/{rows: 99595 (1.71MB), cpu: 1.71M, memory: 0B, network: 0B}

CPU: 408.00ms (87.74%), Scheduled: 1.45s (95.83%), Blocked: 0.00ns (0.00%), Output: 48082 rows (845.19kB)

Input avg.: 750000.00 rows, Input std.dev.: 64.05%

orderstatus := orderstatus:varchar(1):REGULAR

custkey := custkey:bigint:REGULAR

orderdate := orderdate:date:REGULAR

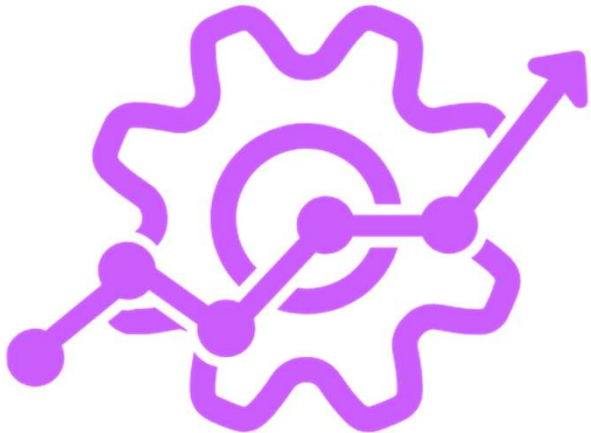
totalprice := totalprice:double:REGULAR

Input: 1500000 rows (41.48MB), Filtered: 96.79%, Physical input: 16.21MB, Physical input time: 1.26s

CBO Optimizations

Two categories of optimizations

- Avoid doing unnecessary work
- Do the work at hand as fast as possible



Optimization examples (always check docs as optimizations vary per connector)

Connector:

- Column pruning (skip reading unneeded columns)
- Predicate pushdown (push filters to data source)
- Partition pruning (skip reading unneeded partitions/files)

Pushdown optimization:

- Aggregation pushdown (push aggregates down to data source)

Joins:

- Join type selection: Partition vs Broadcast
- Join reordering

Predicate pushdown into connectors (*postgres catalog example)

Query (without filter predicate): **SELECT** * **FROM** postgresql.query_plan_with_stats.orders_sf10;

Explain Analyze Plan:

Fragment 1 [SOURCE]

CPU: 34.63s, Scheduled: 1.37m, Blocked 0.00ns (Input: 0.00ns, Output: 0.00ns), **Input: 15000000 rows (1.81GB)**, per task: avg.: 15000000.00 std.dev.: 0.00, Output: 15000000 rows (1.81GB)
Peak Memory: 1.25MB, Tasks count: 1; per task: max: 1.25MB
Output layout: [orderkey, custkey, orderstatus, totalprice, orderdate, orderpriority, clerk, shippriority, comment]
Output partitioning: SINGLE []
TableScan[table = postgresql:query_plan_with_stats.orders_sf10 query_plan_with_stats.orders_sf10]

...
CPU: 34.63s (100.00%), Scheduled: 1.37m (100.00%), Blocked: 0.00ns (?), Output: 15000000 rows (1.81GB)
Input avg.: 15000000.00 rows, Input std.dev.: 0.00%
...

Query (with filter predicate): **SELECT** * **FROM** postgresql.query_plan_with_stats.orders_sf10 **WHERE** orderdate **BETWEEN** **DATE** '1995-01-01' **AND** **DATE** '1995-12-31';

Explain Analyze Plan:

Fragment 1 [SOURCE]

CPU: 5.07s, Scheduled: 24.04s, Blocked 0.00ns (Input: 0.00ns, Output: 0.00ns), **Input: 2275575 rows (281.88MB)**, per task: avg.: 2275575.00 std.dev.: 0.00, Output: 2275575 rows (281.88MB)
Peak Memory: 1.25MB, Tasks count: 1; per task: max: 1.25MB
Output layout: [orderkey, custkey, orderstatus, totalprice, orderdate, orderpriority, clerk, shippriority, comment]

...
TableScan[table = postgresql:query_plan_with_stats.orders_sf10 query_plan_with_stats.orders_sf10 **constraint on [orderdate]**]

...
CPU: 5.07s (100.00%), Scheduled: 24.03s (100.00%), Blocked: 0.00ns (?), Output: 2275575 rows (281.88MB)
Input avg.: 2275575.00 rows, Input std.dev.: 0.00%
...

Hive partition pruning

Explain Analyze Query:

```
EXPLAIN ANALYZE
SELECT orderdate, sum(totalprice) total
FROM glue.bootcamp_explain_plan.orders_orderstatus_partition
WHERE orderstatus = 'F'
GROUP BY orderdate
ORDER BY total DESC
LIMIT 10;
```

Table DDL:

```
CREATE TABLE
glue.bootcamp_explain_plan.orders_ostatus_opriority_partition
(...)
WITH (
    format = 'ORC',
    partitioned_by = array['orderstatus', 'orderpriority'])
```

Fragment 3 [SOURCE]

CPU: 823.19ms, Scheduled: 2.64s, Blocked 0.00ns (Input: 0.00ns, Output: 0.00ns), Input: 729413 rows (9.74MB); per task: avg.: 729413.00 std.dev.: 0.00, Output: 6289 rows (85.98kB)

Peak Memory: 3.95MB, Tasks count: 1; per task: max: 15.65MB

Total split distribution time: 150.99ms

Output layout: [orderdate, sum_0]

Output partitioning: HASH [orderdate]

Aggregate[type = PARTIAL, keys = [orderdate]]

Layout: [orderdate:date, sum_0:double]

Estimates: {rows: 729413 (9.74MB), cpu: ?, memory: ?, network: ?}

CPU: 330.00ms (38.78%), Scheduled: 948.00ms (35.53%), Blocked: 0.00ns (0.00%), Output: 6289 rows (85.98kB)

Input avg.: 145882.60 rows, Input std.dev.: 0.27%

sum_0 := sum(totalprice)

TableScan[table = hive_training:training:orders_partition]

Layout: [totalprice:double, orderdate:date]

Estimates: {rows: 729413 (9.74MB), cpu: 9.74M, memory: 0B, network: 0B}

CPU: 494.00ms (58.05%), Scheduled: 1.69s (63.34%), Blocked: 0.00ns (0.00%), Output: 729413 rows (9.74MB)

Input avg.: 145882.60 rows, Input std.dev.: 0.27%

totalprice := totalprice:double:REGULAR

orderdate := orderdate:date:REGULAR

orderpriority:varchar(15):PARTITION_KEY

:: [[1-URGENT], [2-HIGH], [3-MEDIUM], [4-NOT SPECIFIED], [5-LOW]]

orderstatus:varchar(1):PARTITION_KEY

:: [[F]]

Input: 729413 rows (9.74MB), Physical input: 16.03MB, Physical input time: 634.49ms, Splits: 5

Table contains
15,000,000 rows

Aggregation pushdown

Explain Analyze Query:

EXPLAIN ANALYZE

```
SELECT orderstatus, sum(totalprice)
FROM orders_sf10
GROUP BY orderstatus;
```

Explain Analyze Plan:

Fragment 2 [SOURCE]

CPU: 14.76s, Scheduled: 51.23s, Blocked 0.00ns (Input: 0.00ns, Output: 0.00ns), Input: 15000000 rows (214.58MB); per task: avg.: 15000000.00 std.dev.: 0.00, Output: 3 rows (45B)

Peak Memory: 2.30MB, Tasks count: 1; per task: max: 2.35MB

Output layout: [orderstatus, sum_0]

Output partitioning: HASH [orderstatus]

Aggregate[type = PARTIAL, keys = [orderstatus]]

Layout: [orderstatus:varchar(1), sum_0:double]

Estimates: {rows: 14999213 (228.87MB), cpu: ?, memory: ?, network: ?}

CPU: 561.00ms (3.80%), Scheduled: 558.00ms (1.09%), Blocked: 0.00ns (0.00%), Output: 3 rows (45B)

Input avg.: 15000000.00 rows, Input std.dev.: 0.00%

sum_0 := sum(totalprice)

TableScan[table = postgresql:query_plan_with_stats.orders_sf10 query_plan_with_stats.orders_sf10 columns=[orderstatus:varchar(1):varchar, totalprice:double:float8]]

Layout: [orderstatus:varchar(1), totalprice:double]

Estimates: {rows: 14999213 (228.87MB), cpu: 228.87M, memory: 0B, network: 0B}

CPU: 14.20s (96.20%), Scheduled: 50.67s (98.91%), Blocked: 0.00ns (0.00%), Output: 15000000 rows (214.58MB)

Input avg.: 15000000.00 rows, Input std.dev.: 0.00%

orderstatus := orderstatus:varchar(1):varchar

totalprice := totalprice:double:float8

Input: 15000000 rows (214.58MB), Physical input time: 147.67ms

Join type selection - Partitioned

```
EXPLAIN ANALYZE
SELECT o.orderkey, o.totalprice, c.name
FROM orders o
JOIN customer c
ON o.custkey = c.custkey;
```

Fragment 1 [HASH]

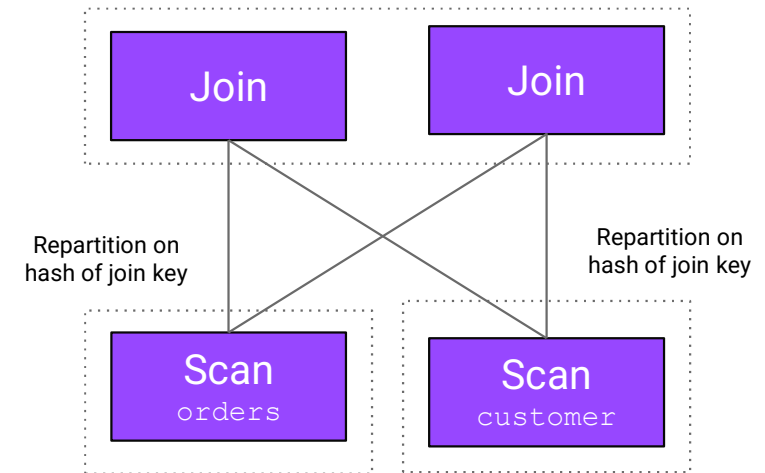
```
...
InnerJoin[criteria = (custkey = custkey_0), distribution = PARTITIONED]
|
| Left (probe) Input avg.: 2343750.00 rows, Input std.dev.: 15.51%
| Right (build) Input avg.: 234375.00 rows, Input std.dev.: 0.22%
| Distribution: PARTITIONED
|
| RemoteSource[sourceFragmentIds = [2]]
| ...
| LocalExchange[partitioning = HASH, arguments = [custkey_0::bigint]]
| ...
| RemoteSource[sourceFragmentIds = [3]]
| ...
```

Fragment 2 [SOURCE]

```
...
Output partitioning: HASH [custkey]
ScanFilter[table = hive:hive_tpch_sf100:orders, dynamicFilters = {custkey = #df_309}]
...
Estimates: {rows: 150000000 (3.77GB), cpu: 3.77G, memory: 0B, network: 0B}/{rows: 150000000 (3.77GB), cpu: 3.77G, memory: 0B, network: 0B}
...
Input: 150000000 rows (3.77GB), Filtered: 0.00%, Physical input: 1.14GB, Physical input time: 59.64s
...
```

Fragment 3 [SOURCE]

```
...
Output partitioning: HASH [custkey_0]
TableScan[table = hive:hive_tpch_sf100:customer]
...
Estimates: {rows: 15000000 (457.76MB), cpu: 457.76M, memory: 0B, network: 0B}
...
Input: 15000000 rows (457.76MB), Physical input: 48.33MB, Physical input time: 6.92s
```



Join type selection - Broadcast

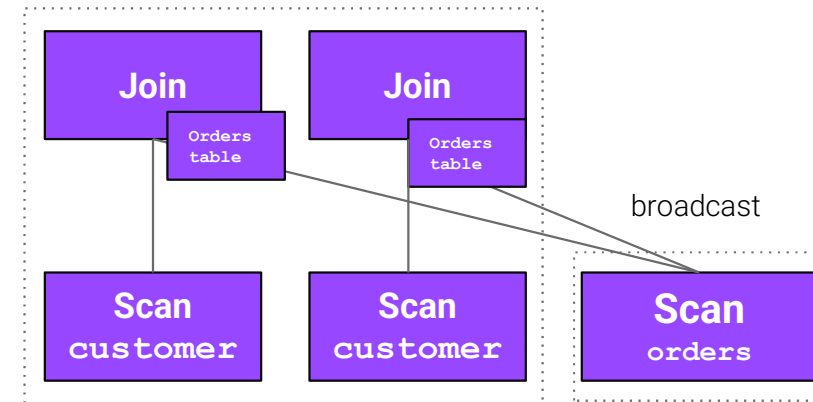
```
EXPLAIN ANALYZE
SELECT o.orderkey, o.totalprice, c.name
FROM orders o
JOIN (SELECT * FROM customer WHERE nationkey = 18) c
ON o.custkey = c.custkey;
```

Fragment 1 [SOURCE]

```
...
InnerJoin[criteria = (custkey = custkey_0), distribution = REPLICATED]
...
Estimates: {rows: 8853512 (346.18MB), cpu: 4.16G, memory: 36.62MB, network: 0B}
...
Left (probe) Input avg.: 1171875.00 rows, Input std.dev.: 97.03%
Right (build) Input avg.: 18737.91 rows, Input std.dev.: 0.70%
Distribution: REPLICATED
...
ScanFilter[table = hive:hive_tpch_sf100:orders, dynamicFilters = {custkey = #df_450}]
...
Estimates: {rows: 150000000 (3.77GB), cpu: 3.77G, memory: 0B, network: 0B}/{rows: 150000000 (3.77GB), cpu: 3.77G, memory: 0B, network: 0B}
...
Input: 150000000 rows (3.77GB), Filtered: 0.00%, Physical input: 1.14GB, Physical input time: 59.15s
...
LocalExchange[partitioning = HASH, arguments = [custkey_0:bigint]]
...
RemoteSource[sourceFragmentIds = [2]]
...
```

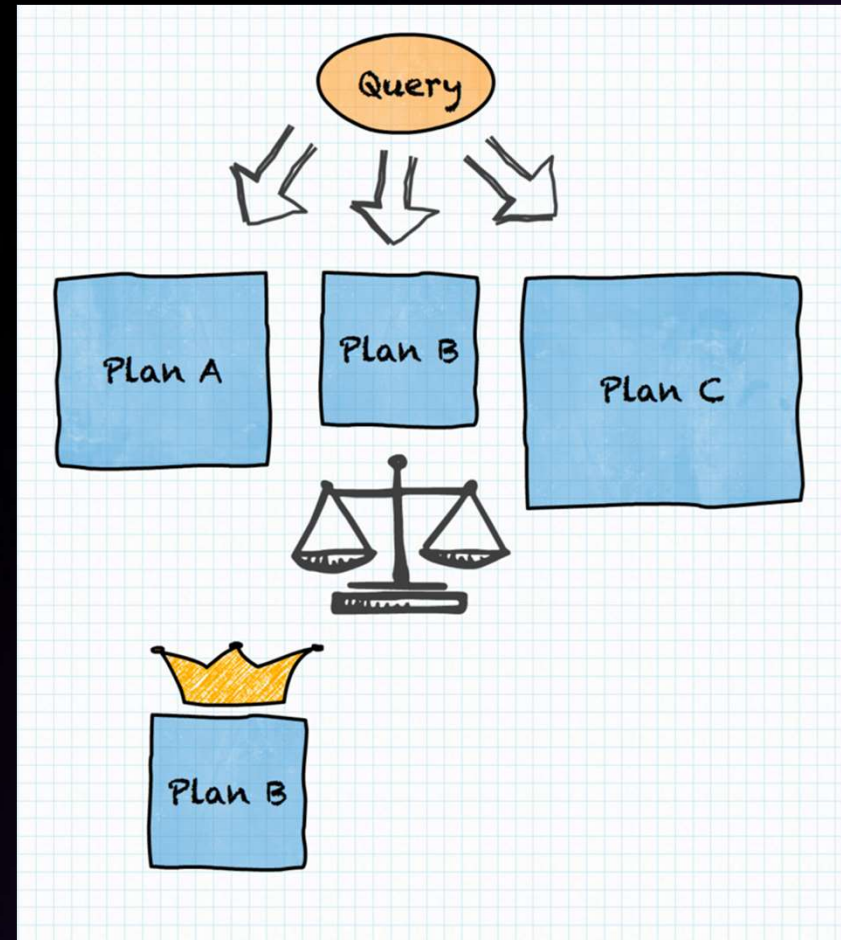
Fragment 2 [SOURCE]

```
...
Output partitioning: BROADCAST
ScanFilterProject[table = hive:hive_tpch_sf100:customer, filterPredicate = (nationkey = bigint '18')]
...
Estimates: {rows: 15000000 (457.76MB), cpu: 586.51M, memory: 0B, network: 0B}/{rows: 600000 (18.31MB), cpu: 586.51M, memory: 0B, network: 0B}/{rows: 600000 (18.31MB), cpu: 18.31M, memory: 0B, network: 0B}
...
Input: 15000000 rows (586.49MB), Filtered: 96.00%, Physical input: 62.49MB, Physical input time: 2.86s
```



Instructor demonstration

Query plan analysis (60 mins)



Hands-on exercises

Lab 1: The EXPLAIN command (20 mins)

Lab 2: The EXPLAIN ANALYZE command (20 mins)

Lab 3: Explore the impact of statistics on query plans (45 mins)

Lesson summary

Cost-based optimization: Query plan analysis

1. Query plans are generated by the system as a processing roadmap. They show each of the steps that Starburst will use to achieve its results. They can be created with, or without, runtime characteristics.
2. Explain plans list stages as fragments which include the pipeline of operations that are parallelized across the workers.
3. Explain plans can be represented visually; similar to flow charts. This is often helpful to understand the stages involved and assess the value of the plan.